

# ML Group 60 - Final Report

GitHub Pages Website for ML Group 60

[View on GitHub](#)

## ML Group 60 - Final Report

[Link to Midterm Checkpoint.](#)

[Link to proposal.](#)

## Introduction & Background

Accurate state estimation is critical for the safety and autonomy of UAVs, particularly in GPS-denied environments. A common approach integrates monocular camera images (visual data) with an inertial measurement unit (IMU) readings to mitigate scale drift and ambiguity [1]. Techniques like VINet use CNN-RNN architectures to map image and IMU sequences to pose estimates [1], while others employ CNNs for visual regression, multiple LSTMs for sequential IMU data, and a centralized LSTM for fusing pose and visual-inertial features [2].

However, fewer studies leverage dynamical data—actuation inputs and external forces—for state estimation in GPS-denied environments. To address this gap, we use the Race Against the Machine Dataset [3], which includes time-synchronized IMU data, RGB frames, motor current (thrust and power), drone speed, ground-truth motion, and command/control inputs (CTBR and motor speeds).

Unlike previous datasets, this dataset provides a complete multimodal representation, enabling dynamics-aided state estimation beyond visual-inertial fusion. It includes high-resolution, high-frequency inertial, visual, input control, and motion capture data [3], making it optimal for evaluating modern UAV estimation frameworks.

Link: [Race Against The Machine Dataset](#)

Note: Although we originally proposed using the VID dataset, we ultimately switched to the Race Against the Machine (RATM) dataset due to several practical constraints. The VID dataset was hosted on a Chinese cloud storage platform with extremely slow and unstable download speeds. Additionally, the data was provided in large .bag files (ROS format), which are difficult to work with on the PACE cluster where ROS is not installed. Python's rosbag library also depends on a working ROS environment, making direct CSV conversion infeasible. Thus, a significant amount of time would have been spent simply downloading and converting the data rather than progressing with

preprocessing or model development. The RATM dataset offered a more accessible, better-documented, and fully synchronized multimodal format, making it more compatible with our pipeline and timeline.

## Problem Definition

UAVs are increasingly prevalent in our daily lives, from fighting wildfires to delivering packages. Such robots estimate their current state  $x_t$  via

$$x_t \approx \hat{g}(x_{t-T}, u_{t:t-T}, y_{t:t-T})$$

where  $u$  are the controls,  $y$  are the sensor measurements, and  $\hat{g}$  is the state estimator [4]. The state estimator is critical as it contains the robot's understanding of its position and velocity; small errors in estimation can lead to unsafe controls.

Unfortunately, finding an analytic expression for  $\hat{g}$  often poses a significant challenge, especially when the system dynamics are unknown. Designing such a function may require weeks of hand-tuning by an expert. Furthermore, estimators tuned for specific conditions may not be robust to disturbances in the dynamics (for example, when a delivery drone carries an unexpectedly heavy package).

To address these limitations, we aim to learn the state estimator from collected data containing sensor measurements (such as IMU and accelerometer data), control inputs (such as motor thrust), and ground-truth trajectory data. To predict the ground-truth measurement of the current state of the drone at a time  $t$ , we will be using the history of sensor measurements and control inputs obtained by the drone and a single initial ground-truth measurement of state of the drone. We will utilize a fixed number of previous timesteps (call this value  $T$ ) to include as inputs to predict the current state. Essentially the initial ground-truth measurement at time  $t-T$ , the control inputs of the drone from timestamps  $t-T$  to  $t$ , and the sensor measurements and analytics of the drone from timestamps  $t-T$  to  $t$  are used in order to predict the state of the drone (including, but not limited to, drone velocity, pitch, thrust, yaw, etc.).

## Methods

### Preprocessing Pipeline:

## 1. Feature selection:

We loaded each synchronized flight log csv file to [pandas.DataFrame](#) and selected timestamps, image file name, elapsed time, IMU data, control data, drone coordinates, drone speeds, and drone rots and residuals. This is because the original data included gate coordinates and marker data which are irrelevant to state estimation.

## 2. Data cleaning:

We utilized [pandas.DataFrame.dropna](#) function to drop rows with missing values to reduce noise from the dataset so that the downstream model can train on complete data.

## 3. Feature Extraction:

We applied a pretrained [ResNet18](#) model to extract a 512-dimensional feature vector for each  $640 \times 480$  resolution image captured by the drone camera for each timestamp. Pretrained on the massive dataset [ImageNet](#), ResNet18 is good at feature extraction and its lightweight made this step fast and efficient.

## 4. Dimensionality Reduction:

We reduced the 512-dimensional visual features for each image to 100 features using [PCA](#), which preserved around 95% variance of the extracted features. Dimensionality reduction is a crucial step in our data preprocessing in order to improve training efficiency as well as reduce noisy features of the data.

## 5. Feature Engineering:

We joined the cleaned IMU, control, and drone state features from step 2 with the visual features extracted and reduced in steps 3 and 4 into unified vectors in order to provide sensor data at each time stamp with visual context.

## 6. Data normalization:

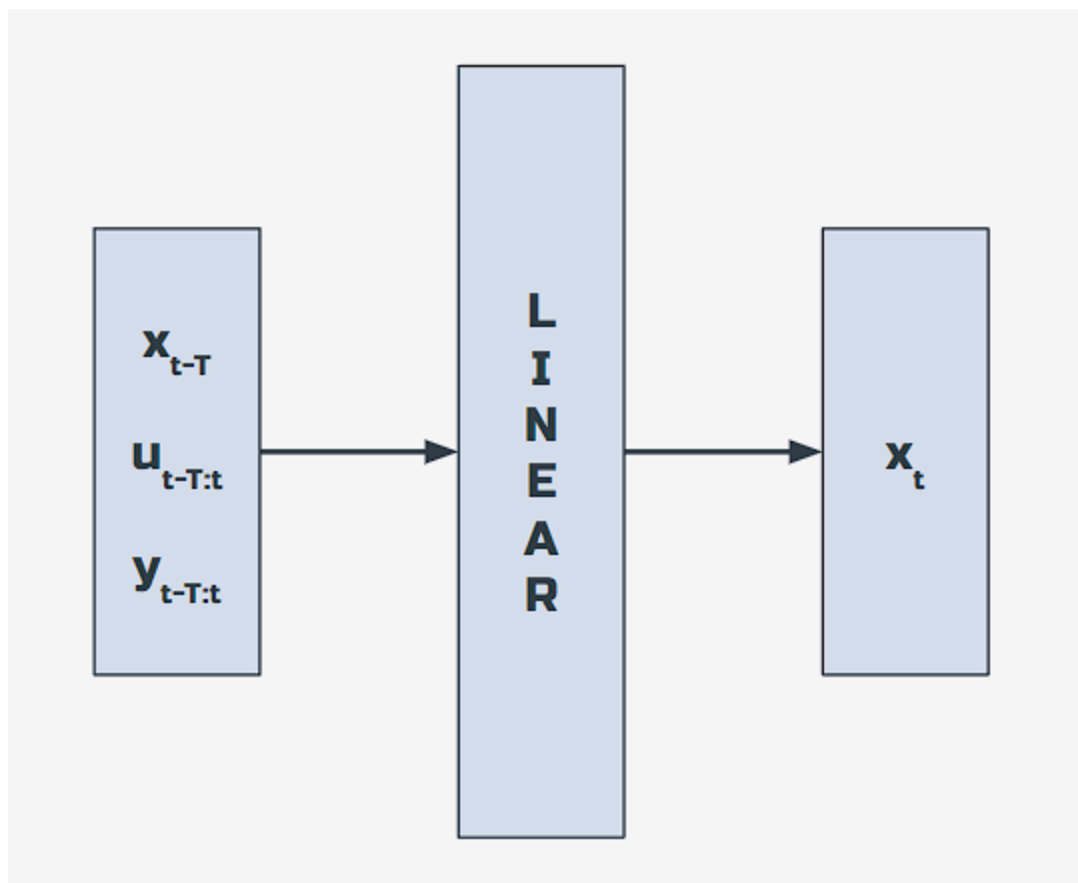
After merging all the data, we used Min-Max Scaling from [sklearn.preprocessing.MinMaxScaler](#) to normalize all the features except the metadata(timestamp, elapsed\_time, img\_filename). The min-max scaling is crucial to normalize all data within the feature to the range [0, 1] and helps maintain stable MSE loss during training and evaluation. It preserves the shape of the original distribution and ensures that all inputs contribute equally to the loss as well as different flight data are normalized with the same parameter.

## Model(s) Implemented:

**Drone State Definition:** We define the drone's state as a 12-dimensional vector, including its x, y, and z positions, roll, pitch, and yaw, as well as its linear and angular velocities.

## 1. Linear Regression:

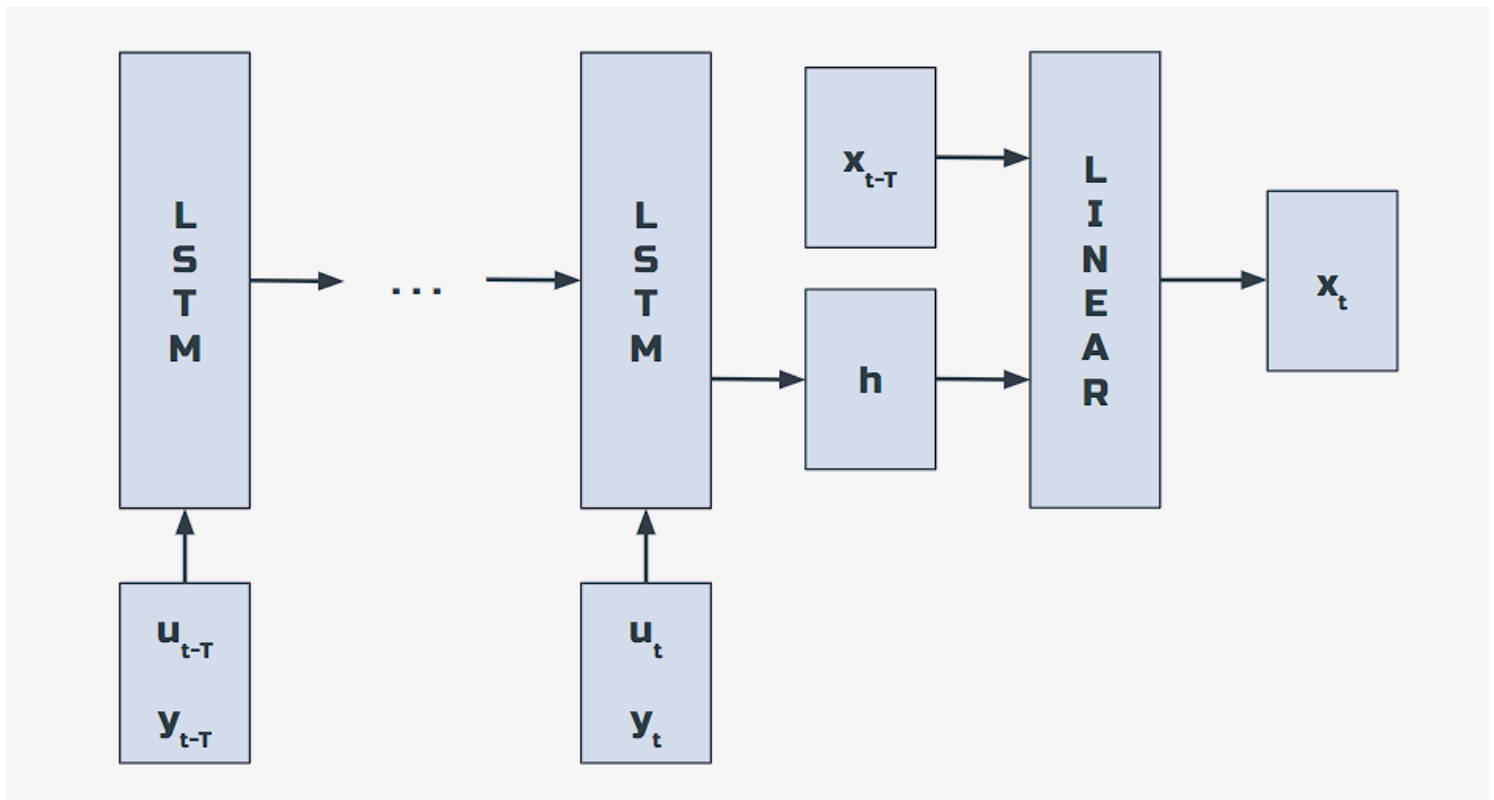
- **Input:** A feature vector created by concatenating control inputs, IMU data, and PCA-reduced image features from the current and previous 49 time steps, along with the ground-truth drone state at time step  $X_{t-50}$ , totaling a 5962-dimensional feature vector
- **Output:** A 12-dimensional output vector describing the drone's state at time  $t$
- **Architecture:** 1 Fully connected layer, lr:  $3e-5$ , epochs: 150
- **Why/Expectation:** We implemented a Linear Regression (LR) model as a simple baseline to evaluate the minimum performance level for our model. The architecture of the model is simple: it consists of a single fully connected layer with no activation function. While we expect the model to perform poorly due to the inherently nonlinear nature of drone state estimation, it helps to act as a point of comparison for evaluating the effectiveness of the MLP and LSTM models



## 2. LSTM Model:

- **Input:** A feature vector created by concatenating control inputs, IMU data, and PCA-reduced image features for each timestep. Repeated for a total of 50 timesteps.

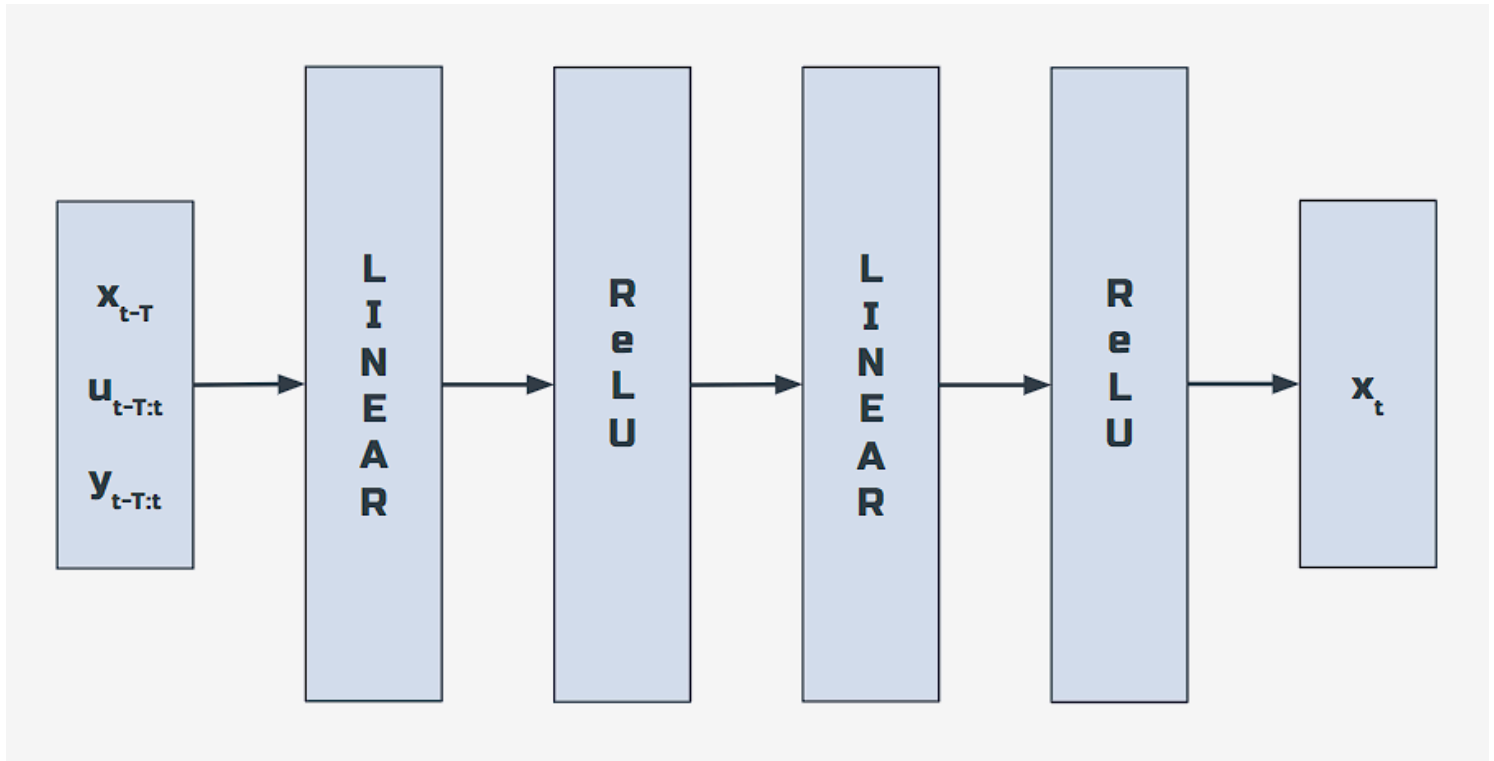
- **Output:** The final hidden state output of the LSTM is concatenated with the ground-truth drone state of  $X_{t-50}$  and fed through a fully connected linear layer with a ReLU activation function to produce the final 12-dimensional output state vector
- **Architecture:** 50 timesteps, each with 119-dimensional input vector, 2 32-dimensional hidden layers, ReLU activation function, lr: 1e-4, epochs: 500
- **Why/Expectation:** Due to the advantage of state estimation being inherently sequential and the fact we have time-synchronized data, we implemented an LSTM model. LSTMs are well-suited for sequential data and time series prediction due to their ability to learn long-range dependencies and retain relevant past information. In addition to this, its ability to capture nonlinear relationships between input features makes this model to seemingly be a perfect candidate for this type of problem. Overall, we believe the LSTM to be a strong baseline in predicting the state estimator for UAV/Drone systems.



### 3. MLP Model:

- **Input:** A feature vector created by concatenating control inputs, IMU data, and PCA-reduced image features from the current and previous 49 time steps, along with the ground-truth drone state at time step  $X_{t-50}$ , totaling a 5962-dimensional feature vector
- **Output:** A 12-dimensional output vector describing the drone's state at time  $t$
- **Architecture:** Two fully-connected hidden layers each of 1024 neurons, ReLU activation function, lr: 1e-4, epochs: 500

- **Why/Expectation:** We implemented the neural network MLP because it effectively captures non-linear relationships between the drone sensor data, control inputs, and ground-truth state variables due to the activation function utilized (as opposed to a linear regression model which can only capture linear dependencies). Given the diverse, high-dimensional nature of our dataset, the MLP provides a strong baseline for evaluating the effectiveness of models in predicting the state of a UAV/drone system.



## Results & Discussion

### Analysis & Comparison of Models

#### 1. LR Model:

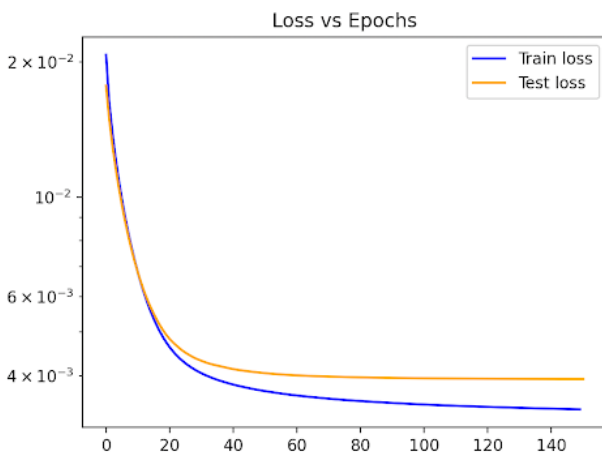
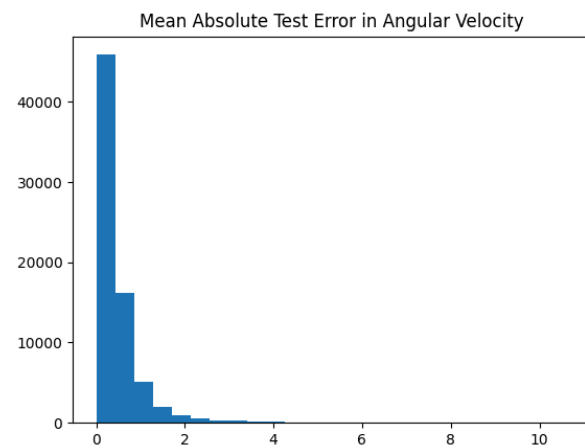
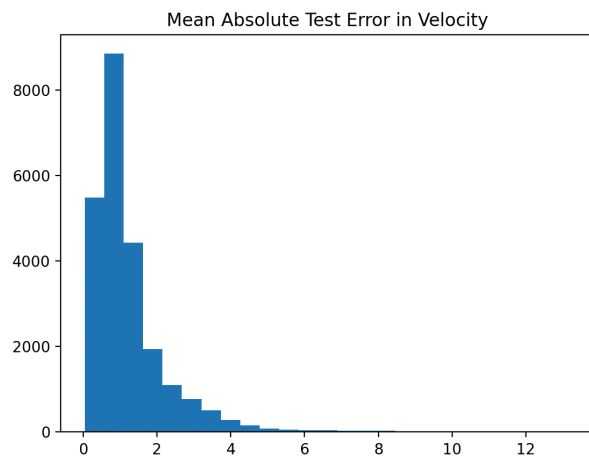
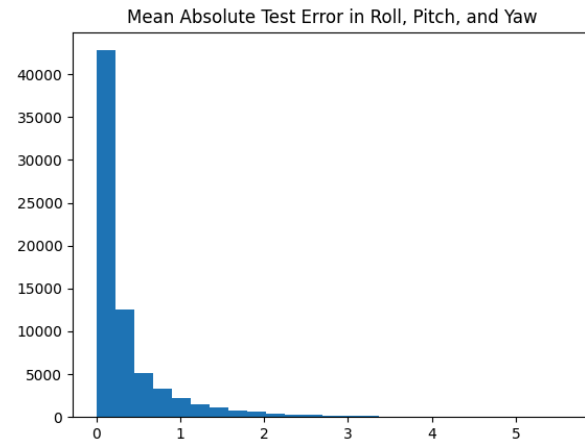
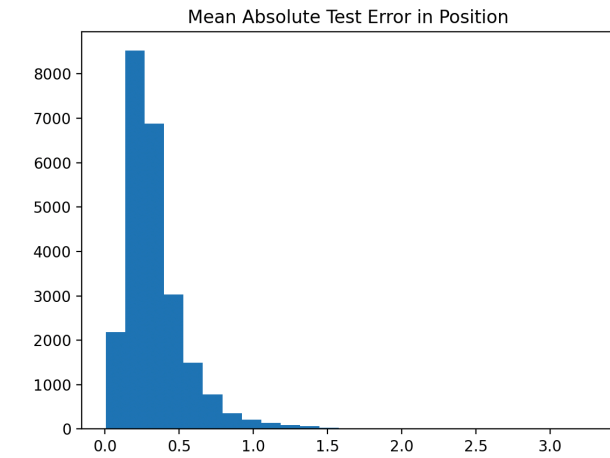
Our hyperparameter tuning yielded the following hyperparameters:

- Learning rate =  $3e-5$
- Number of epochs = 150

Our model achieved the following performance:

- Test loss: 0.00392
- Test MAE: 0.40179
- Test MAE in position: 0.34582 m

- Test MAE in velocity: 1.26146 m/s
- Test MAE in roll, pitch, and yaw: 0.36780 rad
- Test MAE in angular velocity: 0.47012 rad/s



## 2. MLP Model:

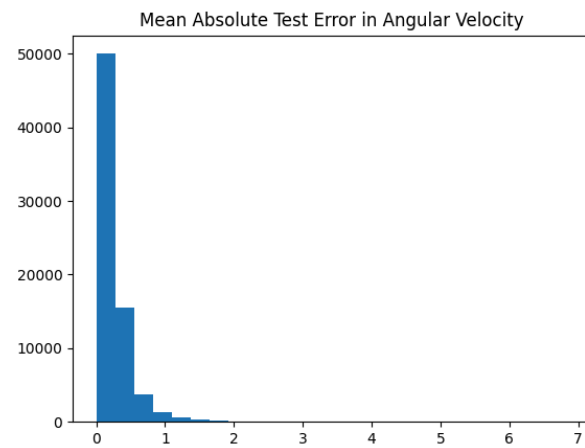
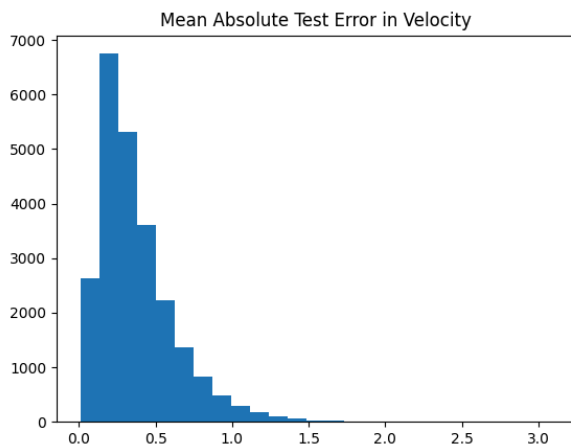
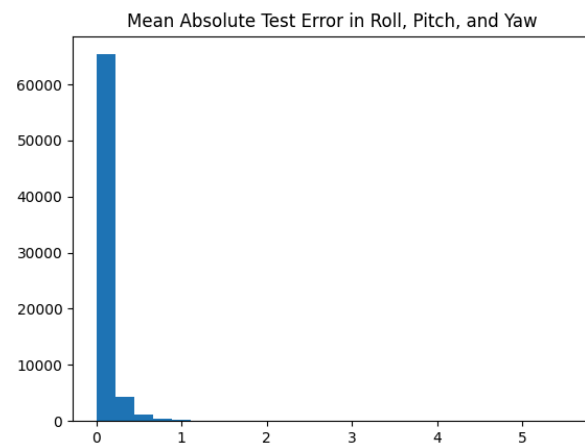
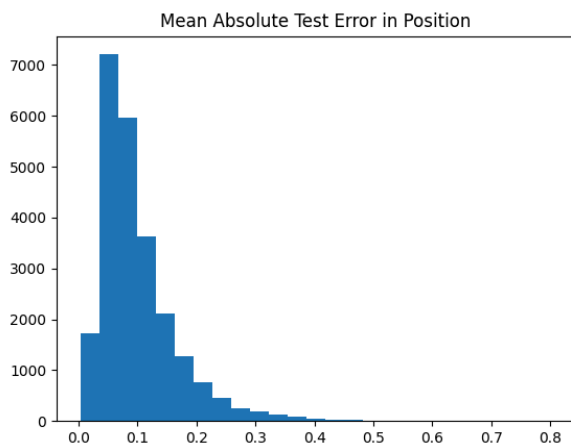
Our hyperparameter tuning yielded the following hyperparameters:

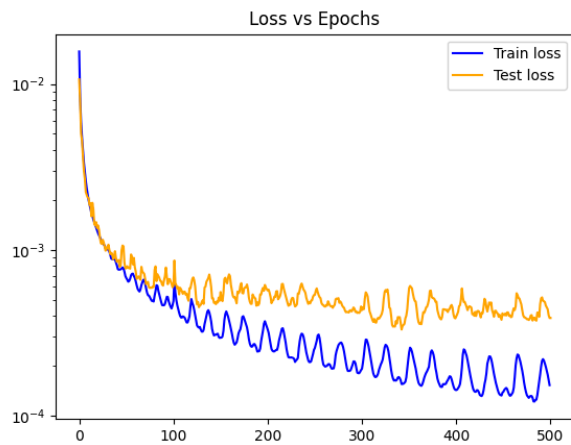
- Hidden layers = [1024, 1024]
- Activation function = ReLU

- Learning rate =  $1e-4$
- Number of epochs = 500

Our model achieved the following performance:

- Test loss: 0.00038
- Test MAE: 0.14124
- Test MAE in position: 0.09942 m
- Test MAE in velocity: 0.37194 m/s
- Test MAE in roll, pitch, and yaw: 0.09596 rad
- Test MAE in angular velocity: 0.24131 rad/s





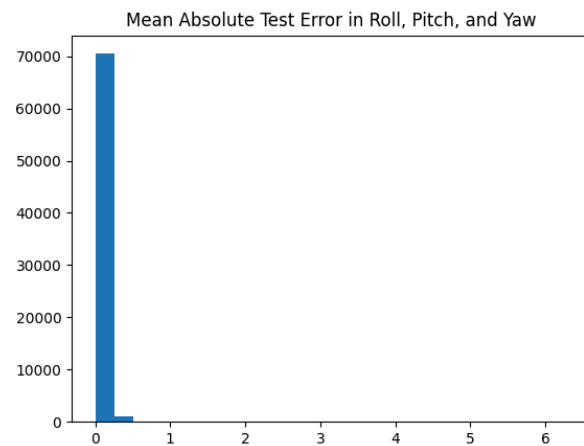
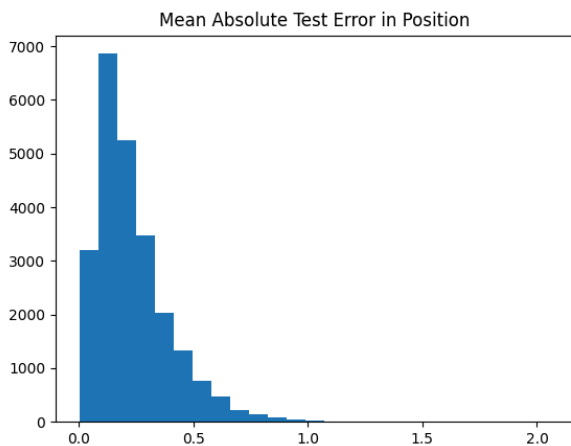
### 3. LSTM Model:

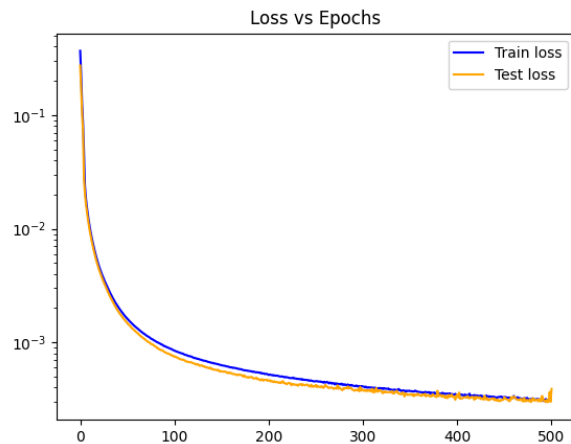
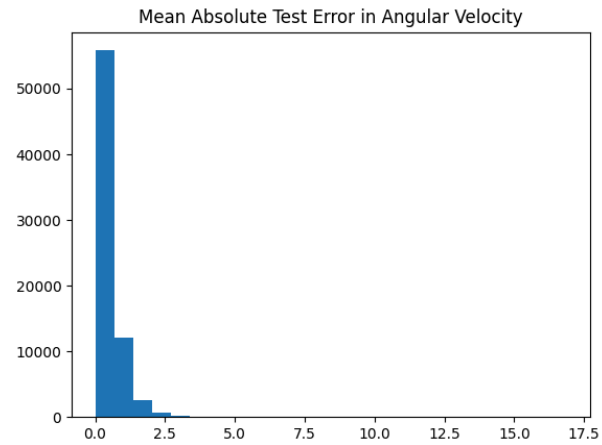
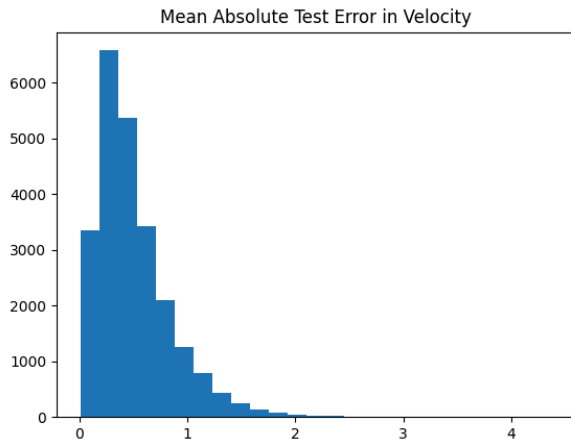
Our hyperparameter tuning yielded the following hyperparameters:

- Lstm hidden vector size = 32
- Lstm number of layers = 2
- Learning rate =  $1e-4$
- Number of epochs = 500

Our model achieved the following performance:

- Test loss: 0.00034
- Test MAE: 0.19813
- Test MAE in position: 0.20432 m
- Test MAE in velocity: 0.46297 m/s
- Test MAE in roll, pitch, and yaw: 0.05387 rad
- Test MAE in angular velocity: 0.43259 rad/s





## Model Comparison

We evaluated the three models on the task of predicting the drone state variables. Their performance, as shown, was assessed using Mean Absolute Error (MAE) and total test loss. (The ground truth in the comparisons refers to the model's prediction from the initial state, not the actual ground truth trajectory. It serves as a reference for evaluating relative model accuracy, rather than an absolute baseline.)

**1. LR Model:** The Linear Regression model, while very lightweight, exhibited the weakest performance across all evaluation metrics. Its simplicity prevented it from capturing the nonlinear dynamics inherent in drone flight, including complex motion patterns and sequential dependencies. The training and test losses converged quickly, but the relatively high loss plateau indicates limited learning capacity. From the histogram plots, LR showed broad error distributions across all predicted quantities, especially in velocity and angular velocity, which highlights its inability to adapt to sharp transitions or dynamic changes in motion. In position and orientation (roll, pitch, and yaw), the model consistently underperformed compared to both the MLP and LSTM, with significantly higher mean absolute errors and a longer tail of extreme values. Despite this, the

Linear Regression model served a valuable role as the baseline, quantifying the minimal performance threshold achievable without nonlinear or sequential modeling.

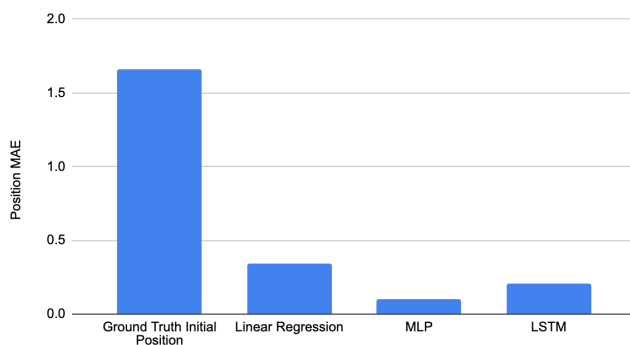
**2. MLP Model:** The MLP model emerged as the strongest overall performer among the three models. The MLP leveraged its deep structure and nonlinear ReLU activations to capture complex relationships in the data. Its test loss converged steadily, even though some fluctuations persisted due to potential overfitting or mini-batch variation. The histograms reveal tight error distributions, with significantly less high-error outliers compared to the Linear Regression model. The MLP also outperformed the LSTM in most metrics, especially in position, velocity, and angular velocity, suggesting that while LSTM's sequential modeling helps with orientation, the feedforward structure of MLP is better suited for high-dimensional, nonlinear mappings. Overall, the MLP was the most effective architecture in this task.

**3. LSTM Model:** The LSTM model, designed to capture temporal dependencies in sequential data, was expected to outperform other architectures due to its capacity to model time series dynamics. The loss curve demonstrates stable training with test loss closely tracking the training loss, indicating minimal overfitting. While the model exhibited strong performance on angular velocity, its histogram reveals a long tail of higher errors, likely due to real-time flight maneuvers or sudden directional changes. Despite this, the LSTM achieved the lowest mean absolute error in orientation, indicating effective modeling of rotational dynamics. However, in position and linear velocity, the distribution of errors is more spread out, suggesting cumulative prediction errors over long sequences. This may reflect the challenge of modeling continuous trajectories. Overall, while the LSTM captured sequential structure well, it slightly underperformed the MLP model in aggregate performance.

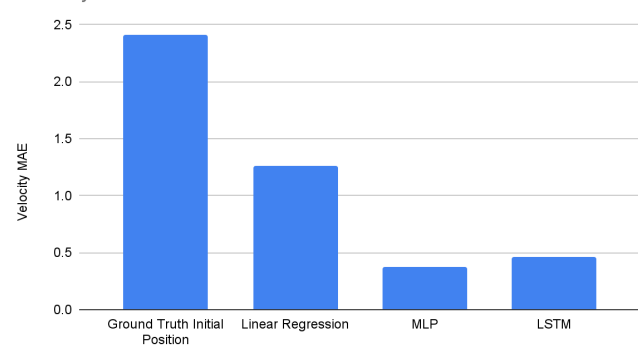
### Summary:

- LR: Underperformed, useful as a baseline.
- MLP: Best overall performance, particularly in position, velocity, and angular velocity.
- LSTM: Strong in orientation prediction, not the best overall.

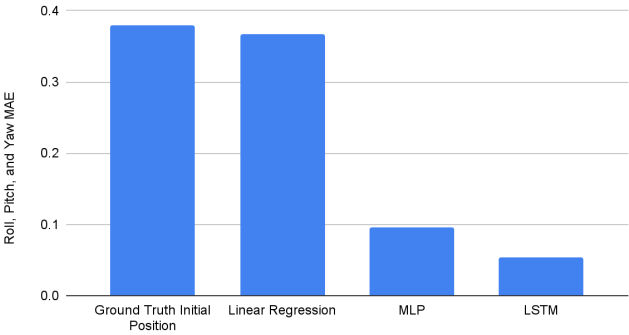
Position MAE



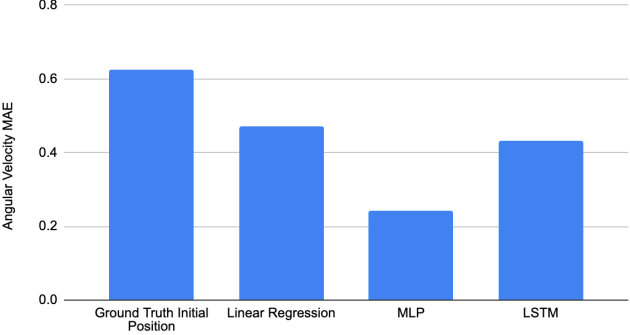
Velocity MAE



Roll, Pitch, and Yaw MAE



Angular Velocity MAE



## Next Steps

- Explore alternate LSTM architectures (e.g., bidirectional or stacked) to better capture temporal dependencies in flight data.
- Perform a comprehensive comparison with classical non-learned state estimators, including Kalman filters, Extended Kalman Filters (EKF), and Unscented Kalman Filters (UKF), particularly when fusing visual and inertial data.
- Evaluate model performance in multi-step forecasting tasks, not just single-step prediction, to assess long-term accuracy and drift.
- Compress or prune MLP and LSTM architectures for efficient deployment on embedded UAV platforms without sacrificing performance.
- Integrate the learned estimator into a real-time UAV simulator and deploy in simulated GPS-denied scenarios to evaluate robustness under occlusion, high-speed maneuvers, and sensor noise.

## GANTT Chart

GAANT Chart

## Contribution Table

# Contribution Table

| Hunter                                        | Tristan                                                             | Xiaomeng                            | Aditya                                               | Ben                                          | Everyone                                                                     |
|-----------------------------------------------|---------------------------------------------------------------------|-------------------------------------|------------------------------------------------------|----------------------------------------------|------------------------------------------------------------------------------|
| MLP, Data Preprocessing, Result visualization | LSTM Implementation, LR Implementation/Training, LSTM Preprocessing | Preprocessing Data, Training models | GitHub Pages, Contribution Table, Final Presentation | LSTM Implementation, LSTM Data preprocessing | Model Implementations, Final Presentation, Training Models, Results analysis |

## Awards

We opt in.

## References

[1] R. Clark, S. Wang, H. Wen, A. Markham, and N. Trigoni, "VINet: Visual-Inertial Odometry as a Sequence-to-Sequence Learning Problem," ariv.org, 2017. <https://arxiv.org/abs/1701.08376>.

[2] F. Baldini, Animashree Anandkumar, and R. M. Murray, "Learning Pose Estimation for UAV Autonomous Navigation and Landing Using Visual-Inertial Sensor Data," *CaltechAUTHORS (California Institute of Technology)*, Jul. 2020, doi: <https://doi.org/10.23919/acc45564.2020.9147400>.

[3] M. Bosello et. al., "Race Against the Machine: a Fully-annotated, Open-design Dataset of Autonomous and Piloted High-speed Flight," arxiv.org, 2024. <https://arxiv.org/html/2311.02667v2#S7>.

[4] T. Barfoot, "4.2.1 Problem Setup," in *State Estimation for Robotics*, Cambridge: Cambridge University Press, Dec. 2022, pp. 96–97

**ML-Project-Group-60** is maintained by **adeb40**.  
This page was generated by [GitHub Pages](#).